Airflow 2 End of Life: Reaffirm or Rethink Your Orchestration Model?

Airflow 2's end of life forces an architectural decision. Here's how to evaluate it.

READ THIS IF YOU'RE A…

DATA ENGINEER
Still on Airflow 2, deciding whether Airflow 3 is worth the migration effort or whether it's time to move to something architecturally different.

CTO / VP ENGINEERING
Making a multi-year platform call as Airflow 2 goes unmaintained, and the cost of the wrong choice is rebuilding team workflows and retraining engineers.

ENGINEERING MANAGER
Managing a team split between upgrading and switching, with real engineering time on the line and delivery timelines that can't absorb a wrong bet.

kestra.

Table of Contents

01 **Introduction** 1

02 **Airflow 3 Changes the Upgrade Scope** 4
What changed
What improved
What got harder
What stayed the same
Why this matters

03 **The Limits of the Airflow Model at Scale** 7

04 **Why Orchestration Is an Architectural Decision** 8
Who orchestration is for
Where code-driven orchestration genuinely wins

05 **The Declarative Alternative** 12
Language-agnostic orchestration
Declarative YAML vs. Python

06 **Where Kestra Enters the Picture** 17
How teams use Kestra
Reusable subflows and workflow composition
Asset management
Event-driven orchestration beyond Cron
Trade-offs to evaluate

07 **The Cost and Shape of Migration** 25
Incremental migration using Strangler Fig

08 **Case Study: Leroy Merlin** 27
The Challenge: A Specialized Skill Gap
The Solution: Declarative Self-Service with Kestra
The Impact: Quantifiable Scaling

09 **Conclusion** 32

10 **About Kestra** 33

SECTION 01

Introduction

Airflow 3 is a better Airflow. But it doesn't change the architecture. For many teams, that distinction won't matter. For others, it's the whole question.

You probably already know the pressure: Airflow 2 reaches end-of-life in April 2026. That deadline forces a decision most teams have been deferring because the question is harder than it looks.

On the surface, it just looks like a question of whether or not to upgrade. However, the actual question is whether to keep betting on this architectural model.

Airflow's design made sense in 2014 when Python dominated data teams and batch pipelines were the primary use case. However, as organizations have grown more heterogeneous, the constraints of that "workflow-as-code" model have become structural bottlenecks:



Organizational bottlenecks

Non-Python teams, from SQL-heavy analysts to infrastructure engineers, cannot self-serve in a system where every pipeline is an executable program.



Technical constraints

The coupled model struggles to coordinate across modern, diverse stacks like AI services, SaaS APIs, and event-driven systems.



Operational scaling

Platform overhead increases faster than business value. When your system must execute Python just to *reason* about an orchestration plan, the design itself becomes the limiting factor.

These aren't incidental issues fixed by a new version but rather outcomes of a model where orchestration logic is tightly coupled to execution. This raises a broader question:

> Should orchestration remain embedded in executable code, or should it exist as a separate architectural layer that coordinates work across systems?

This is where Kestra enters the conversation. While Airflow 3 modernizes the "DAGs-as-code" approach, Kestra was built as a declarative, language-agnostic control plane. By decoupling coordination (YAML) from execution (Python, SQL, etc.), Kestra allows workflows to span languages, teams, and systems without the specialized engineering overhead required by legacy systems.

Different organizations will answer this architectural question differently. Some will reaffirm their current model; others will use this moment to evaluate a more flexible path.

This paper examines the decision between modernizing within the Airflow ecosystem or adopting Kestra's declarative alternative. We explore what truly changes with Airflow 3, where the structural scaling limits of the legacy model appear, and how to evaluate which path best supports your long-term goals.

To help you determine where your team sits, consider the following decision matrix (see next page):

The Airflow EOL Decision Tree

A decision framework for data and platform teams.



SECTION 02

Airflow 3 Changes the Upgrade Scope

For most teams, upgrading to a new version of Airflow has historically meant incremental work: update dependencies, adjust a few operators, move on. Airflow 3 is different.

The transition to Airflow 3 is less a routine upgrade and more a structural refactor. Teams will need to touch workflow code, deployment assumptions, and operational tooling at the same time. Understanding what changed (and what didn't) is mandatory for evaluating how much effort this upgrade actually represents.

What changed

Airflow 3 modernizes the platform with a new React UI, first-class DAG versioning, and improved Assets for explicit data-aware scheduling. Simultaneously, the update involves significant structural refactoring: SubDAGs, context variables, and direct database access are removed, and operators moved to provider packages. The deployment model also has been changed, splitting the webserver into a separate API server and DAG processor, which mandates updates to infrastructure and monitoring setups.

What improved

There are real improvements for teams committed to the Airflow ecosystem. The new UI is significantly more usable, offering faster navigation and clearer debugging, which improves the daily operational experience. Developer ergonomics have improved via the Task SDK, which introduces clearer abstractions and reduces boilerplate. API performance is faster with the FastAPI-based REST API. Finally, DAG versioning and data-aware orchestration (Assets) make dependency modeling more explicit than before.

For Python-centric teams, these improvements may justify the upgrade effort.

What got harder

The migration effort itself is where the scope becomes clearer.

Airflow 3's changes affect entire DAG libraries, not just individual workflows. SubDAG deprecation requires migration to TaskGroups or Assets. Removed context variables mean many templates and macros will need to be rewritten. The REST API v1 deprecation forces updates across external integrations.

Authentication tied to Flask-AppBuilder requires migration or compatibility shims. The Sequential Executor has been removed, requiring teams to adopt alternatives such as the Local Executor even for development environments.

The Airflow team provides tooling, including Ruff linter rules, to identify breaking changes automatically. But detection is not resolution. Each identified issue still requires manual refactoring and validation.

For organizations with large DAG libraries that evolved over years, this is measured in weeks of engineering time, not days.

Infrastructure changes add additional overhead. Splitting the webserver into multiple components affects deployment definitions and monitoring. Moving operators into provider packages requires updates across the codebase. None of this work is conceptually difficult, but it is time-consuming and unavoidable.

This is why many teams describe the transition to Airflow 3 as a controlled rewrite. Over time, teams built significant internal tooling and conventions around Airflow. Updating the foundations means revisiting many of those decisions at once.

What stayed the same

Despite the scope of change, Airflow 3 does not alter Airflow's underlying architectural model.

Workflows remain Python-first. DAGs are still executable code. Orchestration logic is still tightly coupled to execution logic. The scheduler still determines dependencies by executing Python rather than evaluating a static plan. Dynamic behavior remains flexible but difficult to inspect without running the code.

The same scaling dynamics apply. As workflows and teams grow, operational overhead increases non-linearly. Platform teams remain responsible for maintaining reliability, managing custom operators, and absorbing orchestration-specific complexity on behalf of the organization.

Airflow 3 is a better Airflow. It modernizes the platform and improves ergonomics. But it does not introduce a fundamentally different approach to orchestration.

Why this matters

Most teams will invest engineering time to upgrade to Airflow 3 regardless. The question is how that effort is framed internally.

Treating the transition as routine maintenance assumes the existing orchestration model still fits. Treating it as an architectural decision acknowledges that significant refactoring effort is already required, and that this effort creates a window to reassess.

Teams upgrading to Airflow 3 are not only modernizing their tooling. They are reaffirming a model for how workflows are defined, owned, and operated. Understanding that distinction makes it possible to approach the upgrade deliberately rather than treating it as a purely technical exercise.

SECTION 03

The Limits of the Airflow Model at Scale

At a small scale, Airflow's trade-offs are manageable. As you add more workflows across more teams, though, certain constraints start showing up. Here's how three core limitations surface and how they get worse as you scale.

Airflow feature	Limitation	What happens
Python-centric design	Workflows are Python scripts that the scheduler executes. Even simple pipelines need imports, operators, and control flow.	Only engineers who know Python and Airflow internals can own workflows. Every orchestration change becomes a software project.
Orchestration coupled with execution	DAG files are executable programs, not declarations. The orchestration plan doesn't exist until the scheduler runs your code. Dependencies and branching are resolved at runtime, not defined up front.	You can't see what was supposed to run, only what did. Visibility drops as you add workflows. Teams build custom tooling to compensate.
Non-linear operational complexity	Every DAG you add increases scheduler load, parse times, and database contention. Problems don't stay contained to individual workflows but affect the whole system.	You hit performance issues faster than expected. What worked fine with 50 DAGs breaks at 500. Teams hit a ceiling where adding workflows becomes prohibitively expensive.

Apache Airflow is complex and has been challenging to manage, with significant operational overhead.

Senior Engineering Manager, Apple

Read the case study

Airflow's constraints aren't hypothetical. Apple's ML team of 200 engineers evaluated Prefect and Dagster before choosing Kestra as their Airflow alternative. They process massive data volumes across App Store, Apple Music, device logs, and user interactions. The reasons they moved: scalability, language-agnostic YAML (no Python orchestration boilerplate), robust error handling, and cost.

SECTION 04

Why Orchestration Is an Architectural Decision

Once the limits of a specific orchestration tool become apparent, the conversation changes. The question is no longer how to tune configuration or optimize performance. It becomes a question of architecture: what role orchestration plays in the system, how it interacts with execution, and how much organizational behavior it quietly dictates over time.

Orchestration is often treated as scheduling glue. But at scale, it does far more, defining how work is expressed, who can change it, how failures are understood, and where responsibility sits when things go wrong. These are architectural concerns, even if rarely discussed that way.

> The shift from code-driven to declarative orchestration is not a product decision. It is an architectural one, with consequences for team structure, system design, and operational cost that compound over years.

Code-driven (Airflow model)	Declarative (Modern alternative)
Orchestration = executable program	Orchestration = configuration description
Logic embedded in code paths	Logic declared explicitly
Dependencies inferred at runtime	Dependencies visible statically
Flexible but opaque	Transparent but structured
Ownership tied to codebase fluency	Ownership extends to non-technical stakeholders

Airflow embeds orchestration logic inside executable Python code. This prioritizes flexibility and expressiveness, but it couples orchestration to implementation details. As workflows grow and teams diversify, that coupling increasingly defines who can participate and how safely changes can be made.

An alternative approach separates orchestration from execution. Workflows become descriptions of intent in the form of steps, dependencies, conditions, policies defined explicitly rather than inferred from code paths. Execution systems consume that description and perform the work, but the orchestration layer remains inspectable, reviewable, and stable.

Who orchestration is for

When workflows are defined as code, participation is limited to those comfortable working in that codebase and execution environment.

When workflows are defined declaratively, ownership can extend beyond a small group of specialists.

Consider three roles that exist in most organizations running orchestration at scale:

Role	In code-driven orchestration	In declarative orchestration
Platform engineers	Own everything: Airflow deployment, DAG reliability, custom operators, scheduler performance. Become the bottleneck for every team's pipelines.	Own infrastructure: Kestra deployment, RBAC policies, secrets management, monitoring. Set guardrails, but don't own every workflow.
Workflow authors (data engineers, ML engineers, infra teams)	Must write Python, conform to Airflow's execution model, and coordinate with the platform team for deployment.	Define workflows in YAML using their native tools (Python, SQL, dbt, Spark). Deploy independently within their namespace.
Domain consumers (analysts, ops teams, business users)	Depend entirely on platform or engineering teams. Cannot self-serve.	Trigger workflows, view execution status, or use Kestra Apps to interact with pipelines directly.

> *Kestra excels at managing complex ETL workloads. Data engineering pipelines require robust fault tolerance, but configuring them is challenging. We needed a simpler solution.*
>
> **Senior Engineering Manager, Apple**

Read the case study

Where code-driven orchestration genuinely wins

None of this means imperative orchestration is wrong. For small, Python-heavy teams running a handful of tightly coupled pipelines, Airflow's code-first model is genuinely powerful. The ability to express arbitrary logic, use Python's full standard library, and leverage a decade of community-built operators is a real advantage. The gap between prototype and production is short when the entire team speaks Python fluently and the orchestrator does too.

The structural costs described in this paper surface when teams, languages, and workflows diversify. A team of five Python engineers running 50 DAGs will experience Airflow very differently from an organization with 200 engineers running 5,000 workflows across Python, SQL, Spark, and API integrations. Declarative orchestration is designed for the second scenario, not as a replacement for the first.

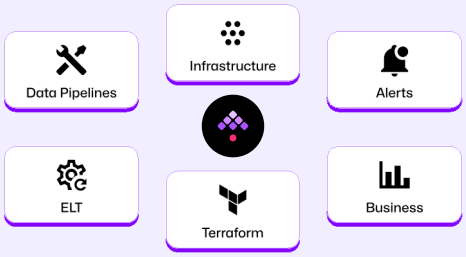
This is the architectural trade-off at the center of the decision facing many Airflow users today. It is not a question of whether one tool has more features than another. It is a question of whether orchestration should continue to be treated as executable logic owned by a few, or as an architectural layer that enables many teams to work safely within a shared system.

SECTION 05

The Declarative Alternative

Once orchestration is treated as architecture rather than just tooling, a different design space opens up. The core question then changes from which orchestrator has the best features to what role orchestration should play in the system at all.

Traditional orchestrators, including Airflow, embed orchestration logic inside executable code. Control flow, dependencies, retries, and conditions are all expressed through a programming language that is evaluated at runtime. This makes orchestration powerful, but it also tightly couples workflow structure to the execution environment and to the language used to define it.



Declarative orchestration

YAML config defines the plan. Any language executes the work. The full workflow is visible before anything runs.



Code-driven orchestration

Python code is the plan. The scheduler executes code to discover what to run. The workflow only exists at runtime.

An alternative approach separates orchestration from execution entirely. In this model, orchestration is expressed declaratively as configuration rather than code. The workflow describes *what should happen* (steps, dependencies, conditions, policies) without embedding *how* each step is implemented. Business logic remains native to the systems that execute it. Orchestration becomes a control layer that coordinates those systems rather than a place where logic accumulates.

Language-agnostic orchestration

Language choice is never just a technical preference. Over time, it decides who can participate, how systems evolve, and how easily new capabilities can be introduced and understood.

In Python-centric orchestration systems, workflows naturally reinforce Python as the center of gravity. SQL transformations, infrastructure commands, API calls, and external services are all wrapped in Python operators or embedded in Python tasks. Even when the underlying work is not Python, orchestration ownership remains with those who understand the Python execution model and the scheduler's behavior.

As platforms mature, this becomes a coordination problem. Modern data and infrastructure stacks span multiple languages and systems: SQL for analytics, Python for modeling, shell and infrastructure tooling for deployment, APIs for SaaS integration, and increasingly, AI-driven services. When orchestration requires everything to be expressed through a single language, teams spend time translating intent into that language rather than expressing it directly.

A language-agnostic orchestrator takes a different stance. Instead of privileging one execution language, it treats tasks as external units of work. A workflow can invoke Python, SQL, dbt, Spark, shell commands, or HTTP APIs directly, without wrapping them in a dominant language. Orchestration coordinates these systems rather than absorbing their logic.

> *Kestra has been a keystone to design complex execution flows while enhancing our Infrastructure as Code best practices. It now empowers our ingestion with Airbyte, transformation with dbt, and our HighTouch reverse-ETL jobs seamlessly\!*
>
> **Antoine Balliet, Senior Data Engineer, Gorgias**

Read the case study

WHY BEING LANGUAGE-AGNOSTIC MATTERS

- ✓ Teams work in their native tools – SQL, Python, dbt, Spark used directly without wrappers
- ✓ Reduces vendor lock-in – workflows not tightly bound to one runtime
- ✓ Future-proofs team structure – new systems added without orchestrator "learning" each one
- ✓ Lowers coordination overhead – each group uses tools they already know

Declarative YAML vs. Python

The most visible difference between these models is how workflows are expressed.

In code-driven orchestration, a workflow is a program. Dependencies are inferred from code paths. Control flow is defined through conditionals and loops. Understanding what will run requires reading and reasoning about executable logic.

In declarative orchestration, a workflow is a description. Dependencies are explicit, conditions are declared, and the structure of the workflow is visible without execution.

PYTHON (AIRFLOW)

```python
from airflow import DAG
from
airflow.providers.snowflake.operators.snowflake
import SnowflakeOperator
from airflow.providers.dbt.cloud.operators.dbt
import DbtCloudRunJobOperator
from
airflow.providers.slack.operators.slack_webhook
import SlackWebhookOperator
from datetime import datetime, timedelta

def notify_failure(context):
    task_id = context["task_instance"].task_id
    error = str(context["exception"])
    SlackWebhookOperator(
        task_id="notify_failure",
        slack_webhook_conn_id="slack_default",
        message=f"Pipeline dbt_snowflake_etl
failed on task {task_id}: {error}",
    ).execute(context)

default_args = {
    "retries": 3,
    "retry_delay": timedelta(seconds=30),
    "on_failure_callback": notify_failure,
}

with DAG(
    dag_id="dbt_snowflake_etl",
    start_date=datetime(2024, 1, 1),
    default_args=default_args,
    catchup=False,
) as dag:
    extract = SnowflakeOperator(
        task_id="extract",
        snowflake_conn_id="snowflake_default",
        sql="""
            COPY INTO staging.orders
            FROM @my_s3_stage/orders/
            FILE_FORMAT = (TYPE = 'CSV'
SKIP_HEADER = 1)
        """,
        retry_delay=timedelta(seconds=30),
    )
    transform = DbtCloudRunJobOperator(
        task_id="transform",
        job_id=366381,
        dbt_cloud_conn_id="dbt_cloud",
        wait_for_termination=True,
        retry_delay=timedelta(minutes=1),
    )
    extract >> transform
```

YAML (KESTRA)

```yaml
id: dbt_snowflake_etl
namespace: company.team

tasks:
  - id: extract
    type: io.kestra.plugin.jdbc.snowflake.Query
    url: jdbc:snowflake://account.snowflakecomputing.com
    username: "{{ secret('SNOWFLAKE_USER') }}"
    password: "{{ secret('SNOWFLAKE_PASS') }}"
    sql: |
      COPY INTO staging.orders
      FROM @my_s3_stage/orders/
      FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1)
    retry:
      type: constant
      interval: PT30S
      maxAttempts: 3

  - id: transform
    type: io.kestra.plugin.dbt.cloud.TriggerRun
    accountId: "{{ secret('DBT_CLOUD_ACCOUNT_ID') }}"
    token: "{{ secret('DBT_CLOUD_API_TOKEN') }}"
    jobId: "366381"
    wait: true
    retry:
      type: constant
      interval: PT1M
      maxAttempt: 3

errors:
  - id: notify_failure
    type:
io.kestra.plugin.slack.notifications.SlackIncomingWebhook
    url: "{{ secret('SLACK_WEBHOOK') }}"
    messageText: "Pipeline {{ flow.id }} failed on task
{{ task.id }}: {{ error.message }}"
```

Yet Another Markup Language (YAML) is ideal for declarative orchestration because of how deliberately simple it is. It's human-readable, diff-friendly, and widely supported. Its limitations are a feature because it encourages clarity over cleverness.

Declarative workflow can be inspected statically. Reviewers see the full execution plan, dependencies, and conditions without running anything. Standard tooling (diffs, linters, version control, review workflows) apply naturally. Changes are easier to audit because intent is expressed directly.

This does not mean declarative workflows are more powerful than code. They trade some dynamic flexibility for transparency and predictability. Highly dynamic behavior that is trivial to express in Python must be modeled more deliberately. But that constraint is intentional, making orchestration understandable, governable, and safer to change as systems grow.

YAML PRIMER

5 BUILDING BLOCKS

`key: value` – basic pairs
Indentation – defines nesting
`- item` – list entries
`|` – multi-line strings
`#` – comments

WHY IT WORKS FOR ORCHESTRATION

YAML is pure data – no functions, no control flow, no imports. Your business logic stays in Python, SQL, or whatever language it was written in. The orchestration layer around it becomes configuration instead of more code.

Learn about YAML

SECTION 06

Where Kestra Enters the Picture

Kestra is an open-source orchestration platform designed around a declarative, language-agnostic architecture. Unlike traditional orchestrators that embed orchestration logic inside executable code, Kestra separates orchestration from execution. Business logic remains in the tools and languages teams already use. Orchestration becomes configuration rather than program flow.

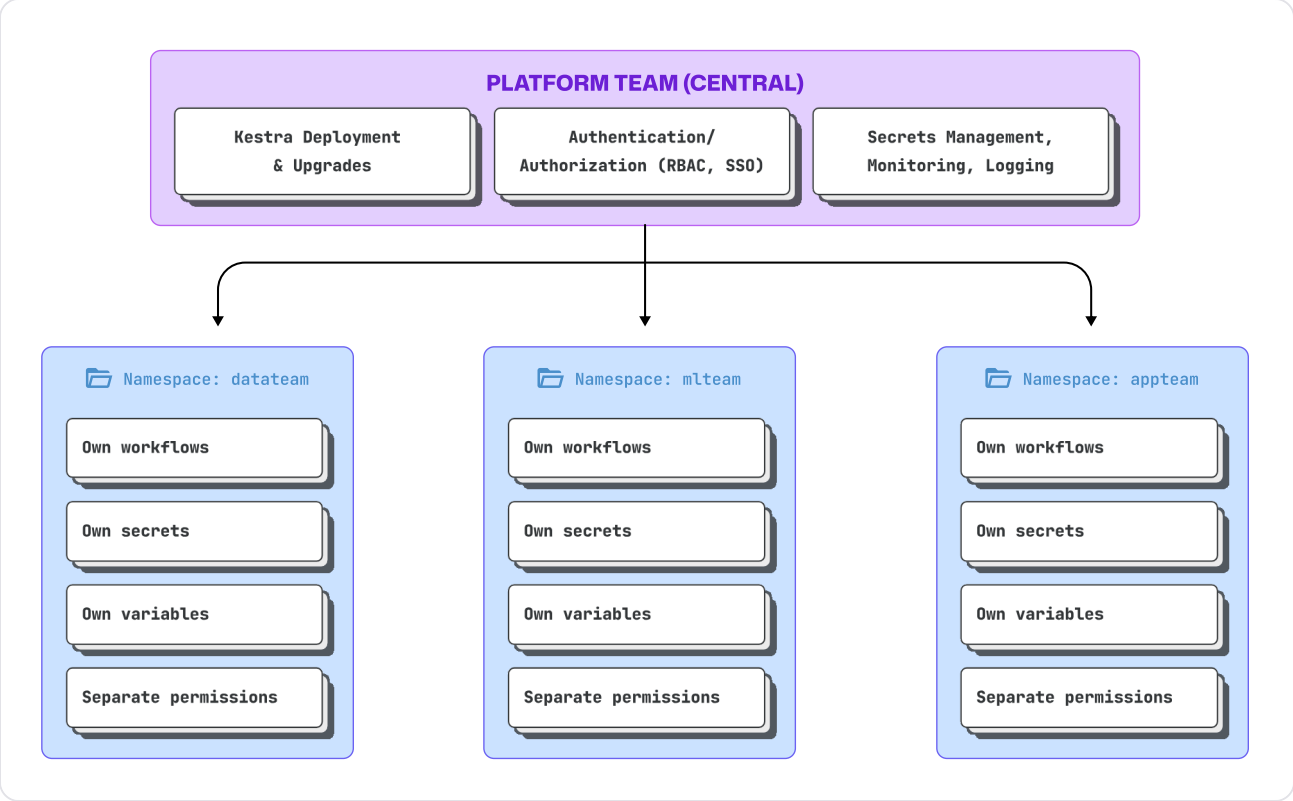
This distinction changes the nature of migration. Moving to Kestra is not about rewriting pipelines in a new framework. It's about translating existing work into an explicit orchestration layer that describes *what runs, when, and why,* without re-implementing the work itself.

> *We needed orchestration built for flexibility and scale, without forcing us to rewrite our existing pipelines. Kestra fit naturally into our stack and enabled our teams to move quickly without disruption.*
>
> **Director of Engineering, Acxiom (120+ engineers, 50+ multi-tenant customers)**

Read the case study

How teams use Kestra

In real-world deployments, Kestra is typically run as a shared orchestration platform with clear separation of concerns between platform ownership and workflow ownership.



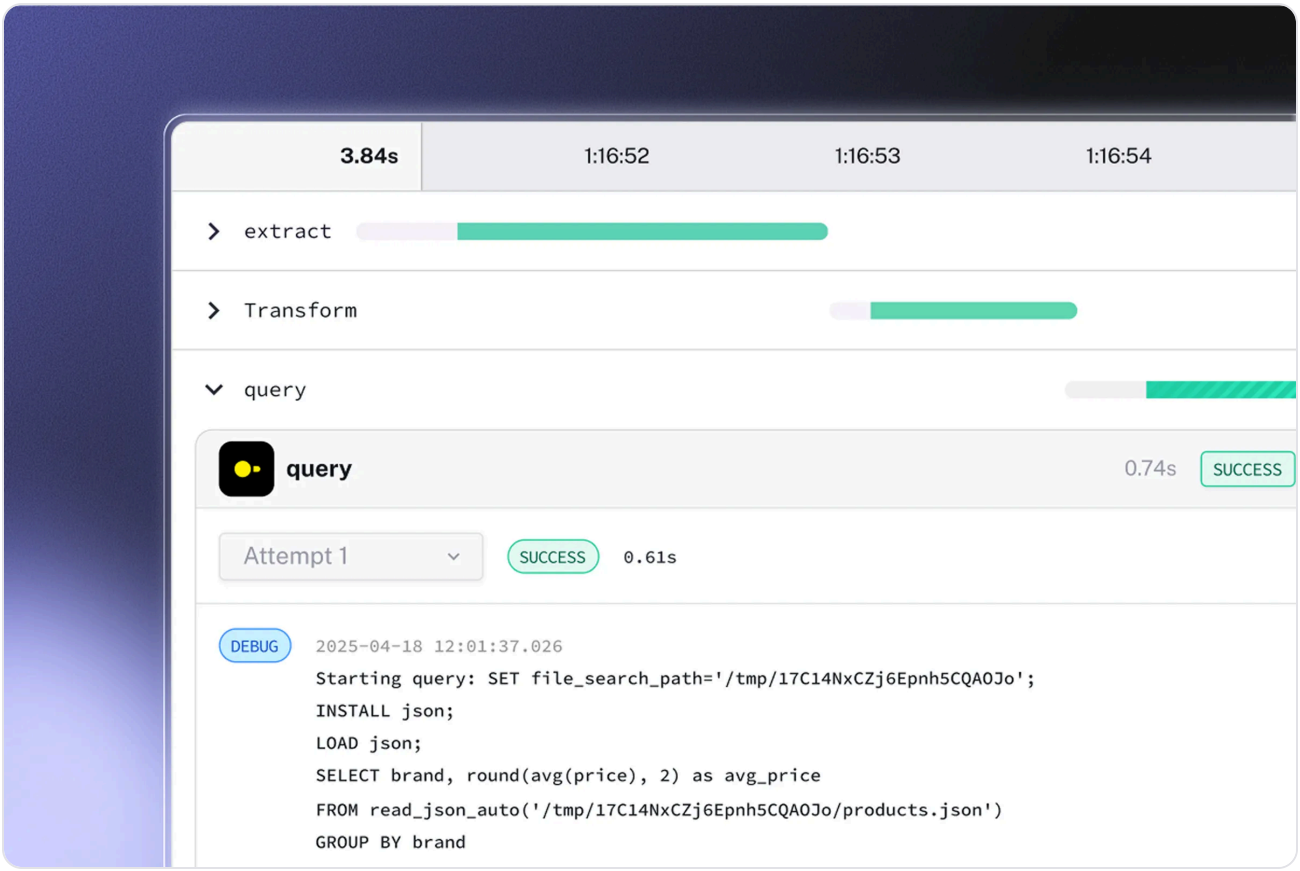
A central platform team manages the core infrastructure: the Kestra deployment, authentication and authorization via RBAC, secrets management (integrated with AWS Secrets Manager, HashiCorp Vault, or Azure Key Vault), logging, monitoring, and upgrade cadence. This team ensures reliability and security, but does not own every workflow.

Individual teams operate within isolated namespaces. Each namespace contains its own workflows, permissions, secrets, and execution history. Teams can deploy and modify their pipelines independently without impacting others. Failures are contained, visibility is scoped, and governance is enforced structurally rather than through informal process.

This namespace-per-team model allows orchestration to scale organizationally. Adding a new team does not increase coordination overhead across the entire system. It creates a new boundary with clear ownership instead of extending an already shared surface area.

> *Kestra has changed how we handle data orchestration. Instead of spending days fixing issues, we now have full visibility and control. As we scale, having a system that grows with us is invaluable.*
>
> **John Kim, IT Lead, FILA (2.5M monthly executions, 2,000+ active workflows)**

Read the case study



Reusable subflows and workflow composition

Another common production pattern is the use of reusable subflows.

Rather than duplicating logic across pipelines, teams define small, composable subflows for common operations such as validation, notifications, retries, or integration steps. These subflows are maintained once and reused across teams and namespaces.

```
# Subflow: Standardized error notification (maintained by platform team)
id: notify_on_failure
namespace: platform.common

inputs:
  - id: workflow_id
    type: STRING
  - id: error_message
    type: STRING

tasks:
  - id: send_slack_alert
    type: io.kestra.plugin.slack.SlackIncomingWebhook
    url: "{{ secret('SLACK_WEBHOOK_URL') }}"
    payload: |
      {
        "text": "❌ Workflow {{ inputs.workflow_id }} failed: {{ inputs.error_message }}"
      }
```

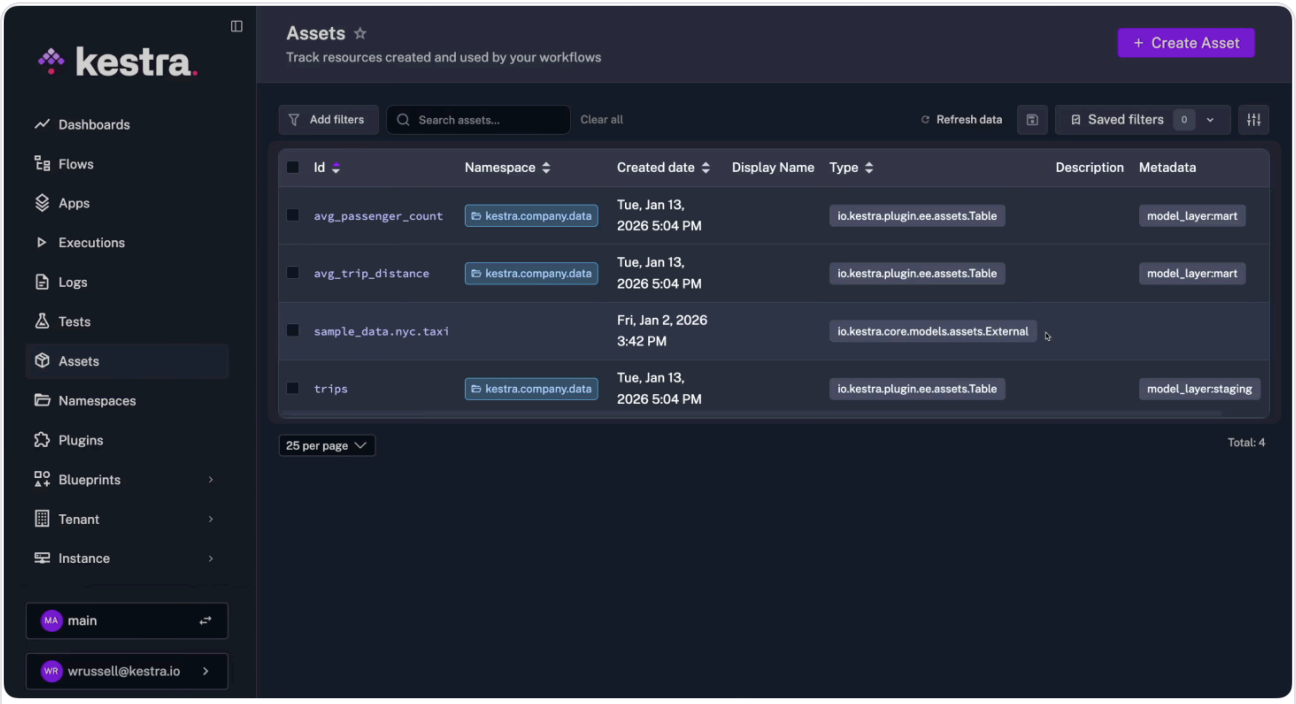
This approach replaces copy-paste reuse with explicit composition. When shared behavior changes, it changes in one place. Platform teams can provide standardized building blocks without owning downstream pipelines, while domain teams retain autonomy over how those blocks are assembled.

Over time, orchestration becomes easier to evolve because shared logic is surfaced intentionally instead of being buried inside workflow implementations.

Asset management

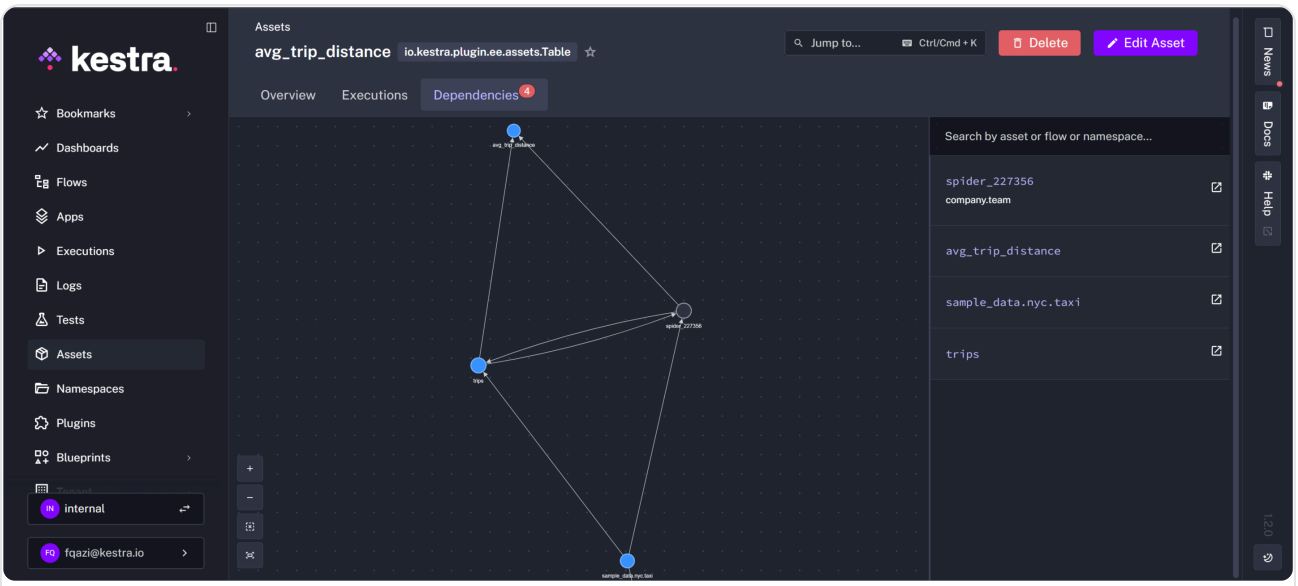
Assets in Airflow 3 allow DAGs to trigger when upstream data is marked as updated. For Python-centric teams with a single platform owner, the model works well. However, because asset definitions live inside DAG code, you still need to know Python to understand what triggers what. Non-engineering stakeholders depend on the platform team to interpret the dependency graph for them, which is manageable at small scale but becomes a coordination bottleneck as cross-team pipelines grow.

Kestra (Enterprise Edition) approaches the problem differently. Instead of treating assets purely as a scheduling primitive, Kestra treats them as a governance layer. Teams declare asset inputs and outputs in YAML at the task level, and Kestra automatically builds a searchable catalog with lineage graphs, execution history per asset, and rich metadata (type, owner, model layer). A data engineer, analyst, and platform engineer all see the same view of what assets exist, what workflows depend on them, and what their current state is, directly in the UI, without executing or reading code.



Asset metadata catalog – searchable, with lineage, execution history, and ownership

On top of that catalog, Kestra offers explicit triggers that respond to asset lifecycle events (created, updated, deleted) and freshness monitoring that detects when assets go stale. These triggers give teams fine-grained control over which events and metadata conditions warrant a downstream run, rather than relying on an implicit "updated" signal.



Asset dependency graph – trace upstream failures without reading code

Where this pays off most is in cross-team failure scenarios. When a downstream pipeline breaks, any team member can trace the dependency chain in the interface without filing a ticket or waiting for a Python engineer to interpret the DAG. Asset lineage becomes a shared operational surface, reducing the quiet coordination overhead that accumulates as organizations grow.

Event-driven orchestration beyond Cron

Teams running Kestra also tend to move beyond purely schedule-driven orchestration.

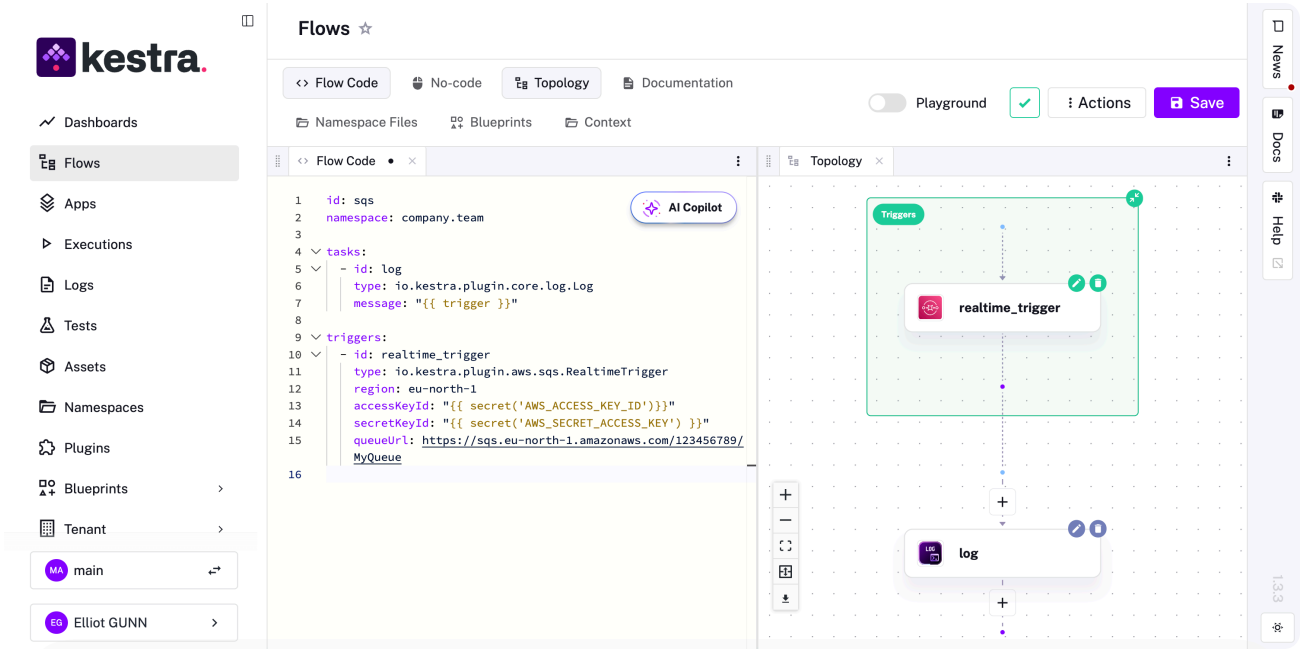
While time-based scheduling remains available, workflows are often triggered by events: a message landing in an SQS queue, a file arriving in S3, a webhook from an external service, or a database change. Kestra's trigger system supports these natively, with configurable conditions, filters, and polling intervals defined directly in the workflow YAML.

YAML TRIGGER DEFINITION

```yaml
id: sqs
namespace: company.team

tasks:
  - id: log
    type: io.kestra.plugin.core.log.Log
    message: "{{ trigger }}"

triggers:
  - id: realtime_trigger
    type: io.kestra.plugin.aws.sqs.RealtimeTrigger
    region: eu-north-1
    accessKeyId: "{{ secret('AWS_ACCESS_KEY_ID') }}"
    secretKeyId: "{{ secret('AWS_SECRET_ACCESS_KEY') }}"
    queueUrl: https://sqs.eu-north-1.amazonaws.com/123456789/MyQueue
```



Kestra's live topology view shows the trigger and task graph as you write YAML – no deploy cycle needed to see what will run

In practice, this reduces unnecessary retries, simplifies dependency management, and shortens end-to-end latency. It also aligns better with modern data and infrastructure architectures, where systems emit signals instead of conforming to centralized schedules.

> *One of our workflows isn't just about moving data from one database to another. It involves reading from a source database, downloading and uploading files across servers, calling multiple APIs, and inserting data into the final destination. Kestra's event-driven model ensures each step happens correctly and automatically retries when issues occur.*
>
> **John Kim, IT Lead, FILA**
>
> **Read the case study**

The result is not the elimination of batch workloads, but a broader orchestration model where schedules are one trigger type among many, rather than the default abstraction for all coordination.

Taken together, these patterns describe an orchestration model that changes how responsibility, visibility, and change are distributed across the organization. The difference is not just how workflows are written, but how teams interact with the orchestration layer day to day.

Trade-offs to evaluate

No platform switch comes without costs, and Kestra is no exception. Understanding where the friction lives helps you plan around it rather than discover it mid-migration.



YAML verbosity

Complex branching that fits in 30 lines of Python may take 80 lines of declarative YAML. Kestra mitigates this with reusable subflows and flow-level variables.



Plugin ecosystem

Kestra's 1,200+ plugins cover major cloud, databases, and data tools, but niche connectors may not exist yet. The plugin SDK makes custom development straightforward.



Community size

Airflow has a decade of Stack Overflow answers and institutional knowledge. Kestra's community is growing (26k+ GitHub stars), and YAML requires no framework-specific expertise.

SECTION 07

The Cost and Shape of Migration

Once orchestration is treated as architecture, migration becomes a cost comparison. For teams already refactoring for Airflow 3, the effort is unavoidable. The question is whether that effort reinforces the same constraints or redirects toward lower long-term operating costs.

> Migration should never be evaluated in isolation. Compare it directly against the ongoing cost of staying on the current model.

Migration is often framed as an all-or-nothing rewrite. That framing is misleading.

Staying (Recurring)	Migrating (Front-loaded)
0.5-2 FTE dedicated to orchestration maintenance	Concentrated effort over defined period
Debugging failures, maintaining custom operators	Assessment, foundation setup, incremental translation
Managing scheduler performance, responding to incidents	Each step reduces dependency on old system
Cost compounds as workflows and teams grow	Cost is directional, not recurring
Unlimited duration	Time-limited initiative with clear trajectory

Most migrations take longer than expected. Many organizations run parallel systems far longer than planned. This reflects production realities, not planning failures. The goal is not to eliminate overlap immediately, but to ensure overlap shrinks rather than becoming permanent.

Framed this way, migration becomes a time-limited initiative with a clear trajectory, rather than an open-ended rewrite that competes indefinitely with feature work.

Incremental migration using Strangler Fig

The Strangler Fig pattern introduces a new orchestration tool alongside Airflow rather than replacing it outright. Both systems run in parallel while migration proceeds selectively.

Simpler, lower-risk pipelines are translated first. More complex or sensitive workflows remain untouched until confidence is established. Where necessary, Kestra can trigger legacy Airflow workflows, allowing orchestration responsibility to shift gradually without forcing immediate rewrites.

This pattern keeps production stable and teams unblocked. Organizations see value before completing migration, not after. Migration progresses when confidence allows, not when deadlines force it.

Eventually, once enough workflows have moved and operational confidence is high, the legacy system can be decommissioned. But that final step only happens when the organization is ready. Until then, parallel operation is an accepted and intentional state, not a failure of planning.

How Kestra works with customers to migrate using Strangler Fig

- Day 1: New workflows built in Kestra from the start
- Phase 1: Simple, low-risk pipelines translated first
- Phase 2: Complex workflows remain in Airflow, triggered by Kestra when needed
- Phase 3: Decommission Airflow once confidence is established
- Production stays stable and feature delivery continues throughout the migration. The organization controls the pace.

AI-assisted translation as a force multiplier

- Airflow DAGs are structured artifacts that AI can analyze mechanically. AI tools generate initial declarative workflow definitions from existing Python code.
- What AI does: Translates task structure, dependencies, and control flow from Python to YAML
- What engineers do: Review generated workflows, confirm correctness, adjust edge cases
- Migration becomes review-driven rather than rewrite-driven, reducing the most tedious parts by 60-70%.

SECTION 08

Case Study: Leroy Merlin

When evaluating whether a declarative, language-agnostic model can operate at enterprise scale, it is essential to look at an organization with massive, decentralized complexity. Leroy Merlin, a global retail leader with over 24,000 employees, provides the blueprint for this architectural transition.

The Challenge: A Specialized Skill Gap

Leroy Merlin's journey toward a cloud-based, serverless infrastructure was initially hindered by legacy tools and specialized skill silos. The tools required manual scaling and were managed through internal ticketing systems that delayed production by days.

While they initially attempted to use Airflow for their Google Cloud Platform (GCP) migration, the platform team ran into a lot of friction, mainly because of three things:

1. **The architecture was unstable**, with simple benchmark tests resulting in failures. This undermined their trust in Airflow.
2. **There was a lot of infrastructure overhead**, with managed services proving too limited and forcing the team to maintain complex Kubernetes clusters that consumed high CPU even when idle.
3. **They were entirely dependent on code**. The code-centric DAGs meant a need to maintain complex Python lifecycles across all source systems, where a single poorly written DAG could lead to bottlenecks that stalled their efforts.

The Solution: Declarative Self-Service with Kestra

Realizing that Airflow couldn't scale to their needs, Leroy Merlin pivoted to Kestra in early 2020. The migration followed a strategic, three-phase path:

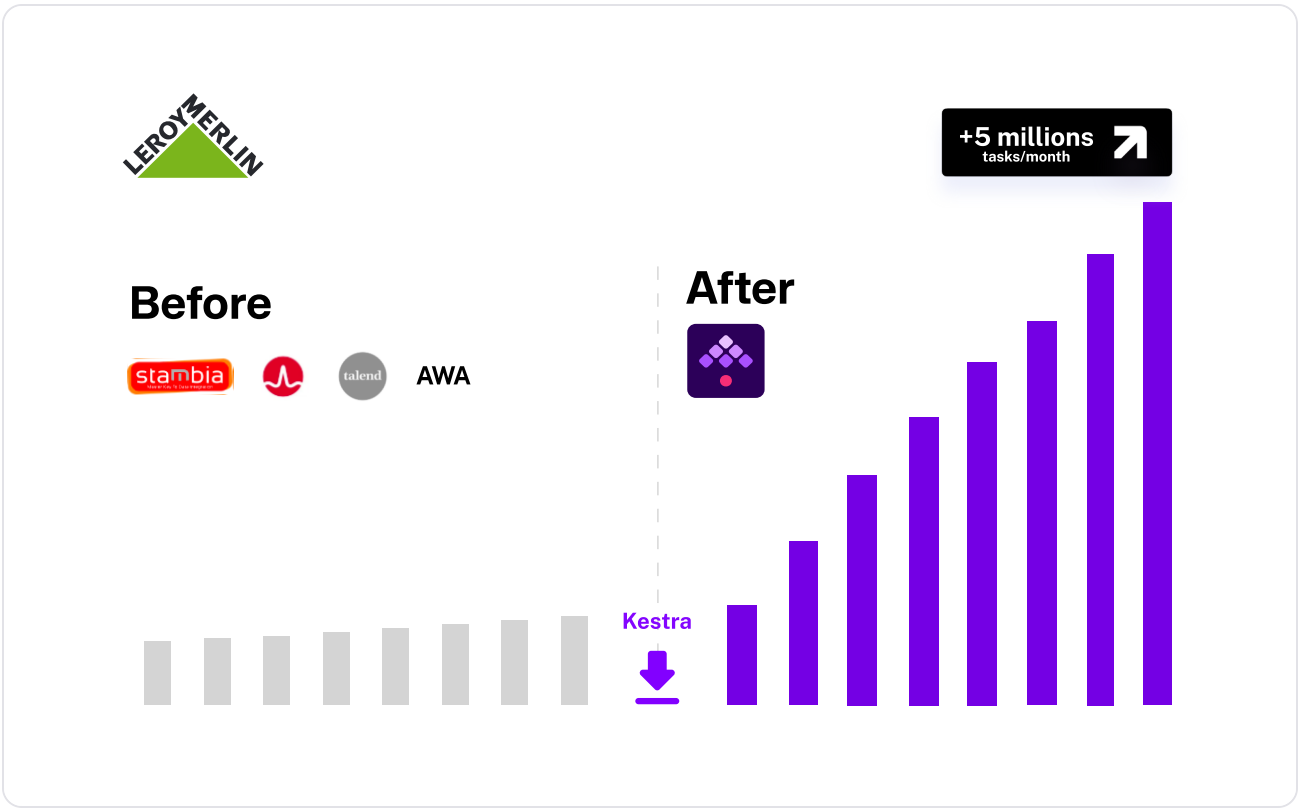
1. They started all new cloud projects directly on Kestra before touching legacy pipelines so that they could prove the technology without disrupting their existing operations.
2. They ran legacy systems on-prem and new cloud-native flows in parallel to bridge the gap between old and new environments while moving data with custom plugins.
3. They used Kestra's SQL pipeline automation to replace the old logic and allow the new event-driven flows to take over the processing load, essentially "shutting off" the legacy orchestrators.

> *Kestra is the first tool that allowed us to develop without installation, use your browser and start to build a true business use case within a few hours. Since the learning curve is easy, you can easily onboard new teammates due to its descriptive language. And, moreover, it handles all parts of the data pipeline: the transport, load, transform, data modeling, data quality and monitoring of all our data pipeline.*
>
> **Leroy Merlin**

Read the case study

The Impact: Quantifiable Scaling

Moving to Kestra allowed Leroy Merlin to decouple data management from the underlying processing tools. By adopting a declarative YAML-based approach, they enabled over **500 data practitioners** (including 80+ data engineers and scientists) to collaborate securely without the overhead of specialized SDK mastery.



The transition also resulted in an almost immediate improvement in Leroy Merlin's structure and operational agility:

01

The volume of high-quality data analytics and production workflows **grew by 900%** over just two years.

02

The descriptive language and browser-based development environment allowed new teammates to **build business use cases within hours**, not weeks.

03

The system handles **400,000+** monthly executions and 3,000,000+ tasks, equivalent to processing 75 days' worth of tasks every single day.

04

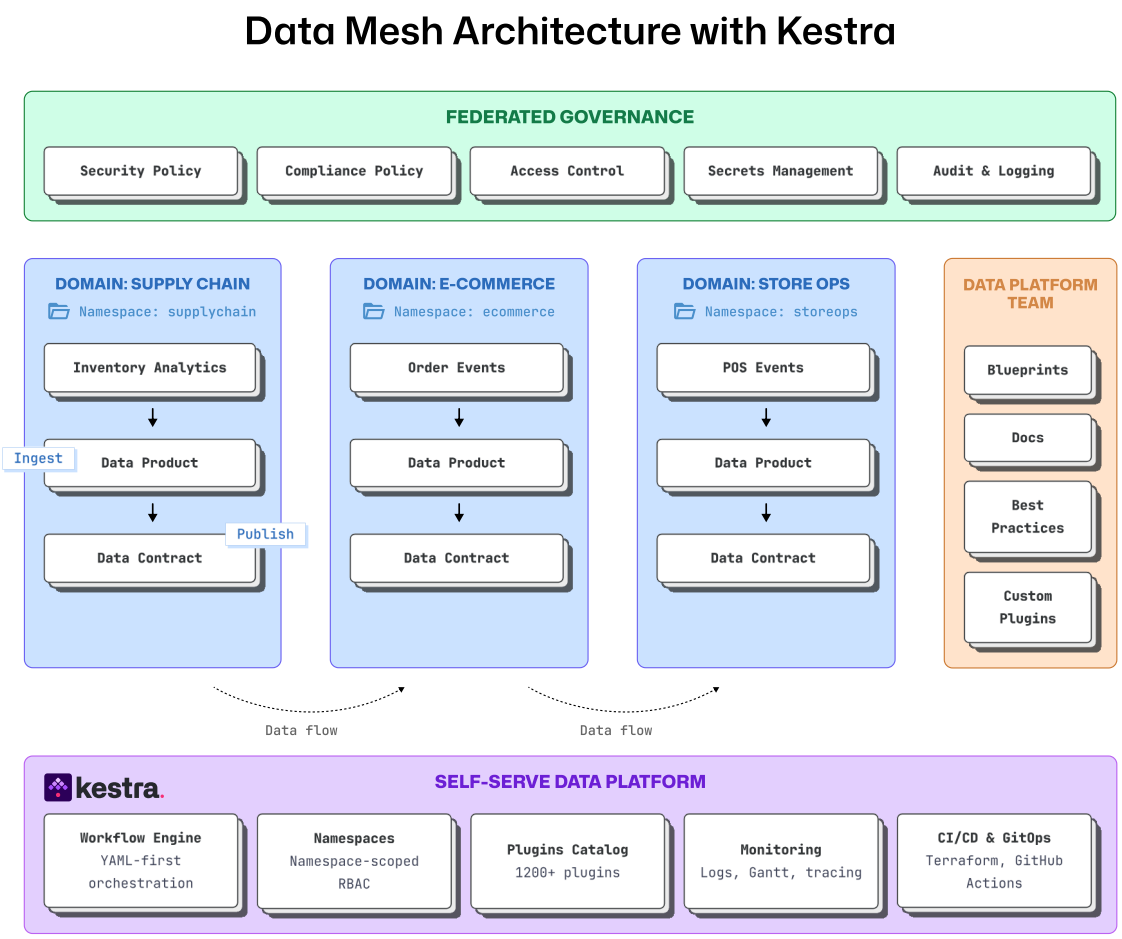
By integrating Kestra with Terraform and GitHub actions, the team was able to realize a full, atomic, CI/CD lifecycle for all of their data resources.

> *Kestra is the unifying layer for our data and workflows. Rather than trying to do it all, it provides a highly open and extensible plugin architecture that connects to everything and allows tasks to be embedded from potentially any application or programming language. You can start small, but then there is no limit to the possibilities and scalability of such an open architecture.*
>
> **Julien Deu, Lead Data Engineer, Leroy Merlin France**

Read the case study

Kestra's control plane replaced Leroy Merlin's fragmented stack with a unified, event-driven architecture. By automating their SQL pipeline creation and technical data validation (via custom plugins), Leroy Merlin freed its data scientists to focus on core business logic rather than data preparation.

Today, the organization runs four distinct environment clusters, proving that a declarative model is not only a simplified alternative, but a requirement for modern, global-scale data architecture.



> *Kestra made the data mesh possible. We produce far more data now, and deliver it nearly 10 times faster."**
>
> **Julien Henrion, CDO at Leroy Merlin, global home improvement retailer with 150,000+ employees across 15 countries**

Read the case study

SECTION 09

Conclusion

Airflow 2's end-of-life is creating a decision window that most organizations haven't had since they first adopted orchestration. The engineering effort required for Airflow 3 is real and unavoidable. The question is what that effort buys.

For teams where Python is the lingua franca, where workflows are owned by a small, tight-knit group, and where the current operational model works, Airflow 3 is a strong path. The improvements are genuine, and the ecosystem is deep.

For organizations where orchestration has grown beyond a single team, a single language, or a single use case, this moment offers something rarer: the chance to treat orchestration as architecture rather than inherited tooling. Declarative, language-agnostic orchestration is not a feature comparison. It is a different answer to the question of how work gets coordinated across systems and teams.

> The organizations that use this window well will not just upgrade their orchestration. They will rearchitect how their teams collaborate, how their systems communicate, and how their platforms scale for the next decade.

Kestra is open source. You can install it in five minutes, run it alongside Airflow from day one, and evaluate the architectural model with your own workflows. No commitment required, no rip-and-replace, no vendor lock-in.

The shift toward declarative orchestration is happening across data engineering, infrastructure automation, and AI/ML operations. Whether your organization moves now or later, understanding the architectural question is the first step.

Try OSS | Talk to sales

SECTION 10

About Kestra

Kestra is an open-source orchestration platform designed around a declarative, language-agnostic architecture. Kestra can be deployed in self-hosted or managed environments and supports multi-team operation through features such as namespace isolation, reusable workflow components, and event-driven execution.

If you'd like additional documentation, technical details, or to try Kestra out yourself:

Try OSS | Talk to sales